Práctica transversal — Docker para Windows y AceriaApp (.NET 8)

Autor del curso: JAIME GALLO

Identificación. Complemento práctico de M01–M08. No es el M09 y no introduce nuevas materias del temario. El alumno utiliza **Windows** como sistema operativo anfitrión, **Docker Desktop + WSL 2 con contenedores Linux** como motor y **PowerShell 7** como terminal. Los proyectos de los ocho módulos se mantienen intactos.

1. Qué hace Docker en cada módulo y por qué

Módulo	Contenedor	Para qué	Dónde corre la API	Evidencia
M01	Ninguno	Introducción sin Docker	Windows + LocalDB	120 pasos y E2E SQL/HTTP
M02–M05	SQL Server Express mediante Testcontainers, *opcional*	Bases aisladas para tests de integración	Windows + LocalDB	xUnit, EF Core real, pruebas de API
M06	RabbitMQ con Docker Compose; SQL Server también puede arrancarse en Testcontainers para tests	Comunicación asíncrona real y outbox/inbox	Producción e Inventario en Windows	mensajes publicados, consumidos y persistidos
M07	RabbitMQ heredado de M06	readiness, fallo, recuperación y resiliencia	Windows	health y flujo tras recuperar broker
M08	RabbitMQ heredado; SQL opcional en Testcontainers para tests	Entrega de eventos detrás de YARP y balanceo	Gateway, Producción e Inventario en Windows	HTTP, proyección persistida, balanceo

Tres escenarios distintos:

1. **Sin Docker:** Windows + SQL Server Express LocalDB, para APIs y tests con `ACERIA_TEST_SQLSERVER` definido (M02–M05).
2. **Docker como infraestructura:** Windows sigue ejecutando las APIs mediante `dotnet run`, y Docker aloja RabbitMQ mediante `compose.yml` (M06–M08). Un contenedor en ejecución no certifica que el evento haya llegado.
3. **Todo en Docker:** laboratorio independiente de las prácticas anteriores. Despliega seis contenedores lógicos: SQL Server Express, RabbitMQ, Producción, Inventario 1, Inventario 2 y YARP. Los contenedores son Linux sobre Docker Desktop; el ordenador y la consola siguen siendo Windows.

2. Preparación verificable del equipo Windows

Requisitos: Windows 10/11 compatible con Docker Desktop y virtualización habilitada, PowerShell 7, SDK .NET 8, Visual Studio con ASP.NET Core y suficiente memoria/disco para SQL Server más cinco procesos. Como orientación práctica, reserva al motor Docker varios gigabytes de RAM y al menos 15–20 GB libres; la cifra efectiva depende del equipo. Para la opción sin Docker instala también SQL Server Express LocalDB.

Instalación y primer diagnóstico (PowerShell 7, ventana de Windows):

```
wsl --status
wsl --update
docker version
docker compose version
docker info --format '{{.OSType}}'
docker run --rm hello-world
dotnet --version
```

Abre **Docker Desktop** y activa el motor **WSL 2 / Linux containers**. La instrucción `docker info --format '{{.OSType}}'` debe devolver `linux`; no significa que el usuario deba instalar Linux como sistema principal. `hello-world` solo prueba que Docker ejecuta una imagen básica; **no** comprueba SQL Server, RabbitMQ ni AceriaApp. Si WSL 2 no está instalado, sigue el asistente oficial de Docker Desktop y reinicia Windows cuando lo solicite. Si `docker version` no comunica con el servidor, no continúes con los ejercicios.

Conceptos antes de ejecutar: una *imagen* es la plantilla inmutable; un *contenedor* es su proceso aislado; *Compose* reúne servicios, redes y volúmenes; un *volumen* conserva SQL y mensajes al reiniciar; el nombre de un servicio Compose es su DNS dentro de la red. `localhost` dentro de un contenedor **no** identifica el PC Windows ni otro servicio.

3. Laboratorio A — SQL Server real para los tests de M02–M05

Este laboratorio **no** despliega la API. Testcontainers 3.10.0 prepara un SQL Server Express temporal y limpia recursos de pruebas. La imagen utilizada por el helper del curso es `mcr.microsoft.com/mssql/server:2022-CU14-ubuntu-22.04`. Se ejecuta en Linux containers sobre Windows.

Desde la **raíz del proyecto terminado** de cada punto, por ejemplo `M02_APIS_REST_ASPNET_CORE/PROYECTO/2.4-documentacion-openapi` (consulta el nombre exacto de la etapa en el repositorio), ejecuta:

```
Remove-Item Env:\ACERIA_TEST_SQLSERVER -ErrorAction SilentlyContinue
docker info --format '{{.OSType}}'
dotnet test .\AceriaApp.sln -c Release
docker ps -a
```

Si `ACERIA_TEST_SQLSERVER` está definida, los tests usan esa conexión en lugar de Testcontainers. La base de prueba es distinta de la base didáctica de la API; comprobar un `dotnet build` no prueba persistencia. Durante la ejecución observa los contenedores con `docker ps` desde una segunda terminal. Comprueba en la salida de xUnit que se ejecutaron y aprobaron todos los tests. Ryuk/Testcontainers gestiona los contenedores temporales; no los elimines manualmente mientras haya pruebas vivas.

Alternativa sin Docker (LocalDB, Windows):

```
sqllocaldb start MSSQLLocalDB
$env:ACERIA_TEST_SQLSERVER = 'Server=(localdb)\MSSQLLocalDB;Database=master;Integrated
Security=True;TrustServerCertificate=True'
dotnet test .\AceriaApp.sln -c Release
```

En ambos casos el proveedor sigue siendo SQL Server. No cambies a SQLite o `InMemory` para fabricar un resultado correcto. Repite la prueba con Docker apagado: la alternativa LocalDB debería funcionar si la instancia está instalada y sana; la variante Testcontainers debe fallar claramente y no aprobar sin motor.

4. Laboratorio B — RabbitMQ con Docker, aplicaciones en Windows

Este es el despliegue **real y mínimo** utilizado desde M06. En el punto 6.1, las prácticas construyen `compose.yml`; M07 y M08 lo heredan. El fichero define `rabbitmq:4.1.5-management`, credenciales didácticas `curso / curso-local`, puertos AMQP `5672` y consola `15672` publicados solo en `127.0.0.1`, y volumen `rabbitmq-datos`.

Desde la carpeta de la etapa (ejemplo final `M08_API_GATEWAY/PROYECTO/8.4-balanceo-carga`):

```
Test-Path .\compose.yml
docker compose -f .\compose.yml config --quiet
docker compose -f .\compose.yml up -d rabbitmq
docker compose -f .\compose.yml ps
docker compose -f .\compose.yml exec -T rabbitmq rabbitmq-diagnostics -q ping
Test-NetConnection localhost -Port 5672
Test-NetConnection localhost -Port 15672
```

El diagnóstico del broker debe devolver código cero, los dos puertos deben estar disponibles y la interfaz `http://localhost:15672/` debe permitir entrar con credenciales **solo de laboratorio**. En **cada** PowerShell en que arranques Producción o Inventario configura:

```
$env:ASPNETCORE_ENVIRONMENT = 'Development'
$env:Comunicacion__RabbitMqUri = 'amqp://curso:curso-local@localhost:5672/'
$env:Comunicacion__WorkersActivos = 'true'
$env:Comunicacion__Prefijo = 'aceria-m08-manual'
```

Completa asimismo las cadenas LocalDB detalladas en M06–M08. Ejecuta `dotnet run --project src/AceriaApp.Inventario.Api --no-launch-profile` y `dotnet run --project src/AceriaApp.Api --no-launch-profile` en terminales distintas, estableciendo puertos y demás variables **del punto correspondiente**. En M08 arranca también `src/AceriaApp.Gateway` y la segunda instancia de Inventario como describe el recorrido 8.4. Las aplicaciones no están en Docker: para ellas `localhost:5672` señala al broker expuesto en el host Windows.

Prueba de fallo real: tras comprobar la entrega outbox/inbox, detén el broker mediante `docker compose stop rabbitmq`, observa readiness degradada y recupera con `docker compose start rabbitmq`. No utilices `down -v` si quieres preservar las colas, las bases o las pruebas de reinicio. `docker compose logs --tail 80 rabbitmq` ayuda a diagnosticar; el estado `Up` por sí solo no acredita mensajería.

5. Laboratorio C — despliegue integral de M08 con Docker Compose

Propósito: aprender la diferencia entre usar Docker solo como proveedor de infraestructura y contenerizar Producción, Inventario y YARP. Este laboratorio reutiliza **sin cambiar** la etapa `M08_API_GATEWAY/PROYECTO/8.4-balanceo-carga`. Los componentes reales son:

- `sqlserver`: SQL Server 2022 Express, puerto **1433 privado** en la red Compose, volumen `sql-datos`.
- `rabbitmq`: AMQP, gestión web y volumen `rabbit-datos`.
- `inventario-1` y `inventario-2`: dos instancias del proyecto `AceriaApp.Inventario.Api` sobre **una misma base**.
- `produccion`: `AceriaApp.Api` con las bases aisladas de API/Hexagonal/DDD y su publicador outbox.
- `gateway`: `AceriaApp.Gateway` (YARP), único puerto público de negocio: `127.0.0.1:5100`.

Atención al cambio técnico: `(localdb)\MSSQLLocalDB` es solo Windows y no funciona dentro de estos contenedores Linux. Por eso, en este laboratorio se conectan las aplicaciones a `Server=sqlserver,1433` con autenticación SQL de laboratorio; esta diferencia se realiza mediante variables

`ConnectionStrings__*` de Compose **sin modificar el código C#**. En la red Docker, `rabbitmq:5672` reemplaza `localhost:5672` y los destinos YARP pasan a `http://inventario-1:8080/`, `inventario-2` y `produccion`. El código sigue apuntando a los mismos contratos, no a nuevos ejemplos.

5.1 Preparar parámetros y construir las imágenes

Desde la raíz del repositorio, entra en `PRACTICA_TRANSVERSAL_DOCKER_WINDOWS`:

```
Set-Location .\PRACTICA_TRANSVERSAL_DOCKER_WINDOWS
Copy-Item .\.env.example .\.env
notepad .\.env
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml config --quiet
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion preparar
```

Cambia la clave del ejemplo de `.env` por otra robusta y **no publiques ese archivo**. La conexión SQL tiene cifrado activado, pero confía en el certificado autofirmado solo para laboratorio. `.env` se usa en Compose; no sustituye a los archivos de configuración del proyecto.

El `Dockerfile.Servicio` compila con `mcr.microsoft.com/dotnet/sdk:8.0`, publica el proyecto con `dotnet publish` y lo ejecuta sobre `mcr.microsoft.com/dotnet/aspnet:8.0`, sin SDK en la imagen final. El argumento de construcción selecciona `AceriaApp.Api`, `AceriaApp.Inventario.Api` o `AceriaApp.Gateway`. El contexto es la **raíz del repositorio**, por lo que están disponibles todas las referencias entre proyectos.

5.2 Iniciar paso a paso y ejecutar pruebas HTTP

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion levantar
```

El script inspecciona Docker, valida Compose, arranca primero SQL y RabbitMQ, espera a que SQL responda de verdad y levanta **secuencialmente** Inventario 1, Inventario 2, Producción y Gateway. Esta secuencia evita que las dos instancias creen simultáneamente las tablas de Inventario. Después comprueba readiness de los backends, liveness del gateway, consulta de stock, crea una orden DDD de dos líneas, planifica por HTTP, exige `202`, exige `409` al repetir, espera la proyección asíncrona y contrasta los datos después de reiniciar los procesos. Cualquier diferencia debe devolver error, no un falso PASS.

Repite las comprobaciones sin reconstruir:

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion verificar
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml ps
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml logs --tail 100 produccion inventario-1
inventario-2 gateway
```

Las respuestas de `/health/ready` validan dependencias (SQL y AMQP); `/health/live` del gateway solo valida el proceso. Un `200` de liveness no demuestra que YARP pueda llegar a Inventario. El stock `Acero` de una base nueva debe ser 300 y los eventos llegan de forma asíncrona, por lo que hay que esperar y consultar su proyección.

5.3 Persistencia, recuperación y limpieza controlada

Los datos SQL y mensajes RabbitMQ sobreviven a `restart` y `stop` gracias a los volúmenes. El script comprueba la recuperación de un identificador concreto. Para detener sin borrar:

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion detener
```

Para eliminar **contenedores y red** pero conservar datos:

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion bajar
```

Solo para **reiniciar de cero los datos didácticos** (destrucción intencional de ambos volúmenes):

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion limpiar -ConfirmarBorradoDatos
```

Nunca ejecutes esa última operación cuando desees demostrar persistencia o reentrega. El broker no debe reutilizarse simultáneamente con el Compose de M06–M08 en el mismo puerto 5672; elige un entorno activo y detén el otro antes de comenzar.

6. Diagnóstico orientado a causas, no a atajos

Síntoma	Causa probable	Acción verificable
`docker version` no conecta	Docker Desktop parado o motor equivocado	Arrancar Docker Desktop; comprobar `OSType=linux`
puerto `5672` ocupado	dos laboratorios RabbitMQ activos	`docker ps --format "table {{.Names}}\t{{.Ports}}"` y detener el que sobra
SQL no llega a `healthy`	memoria, imagen o credenciales; herramientas SQL ausentes	`docker compose logs sqlserver` y revisar prueba de salud; no sustituirlo por InMemory
contenedor arranca y cae	error .NET al inicializar SQL	`docker compose logs produccion` / `inventario-1` y verificar SQL healthy
API dentro de Docker apunta a `(localdb)`	no se configuró `ConnectionStrings__*`	comprobar `docker compose config` sin publicar contraseña
error RabbitMQ desde contenedor	`localhost` en vez de DNS `rabbitmq`, usuarios/puertos	revisar `Comunicacion__RabbitMqUri` y `rabbitmq-diagnostics ping`
gateway devuelve `503`	destinos sin salud o YARP aún sin resolver	verificar `/health/ready` en cada backend y variables ReverseProxy
el evento no aparece	workers desactivados, prefijo distinto o outbox pendiente	revisar `WorkersActivos`, `Prefijo`, broker, logs y proyección
`dotnet test` pasa sin SQL real	test sustituido por proveedor falso	comprobar proveedor SQL Server, evidencias y ausencia de tests omitidos

Criterio de cierre: el Markdown se contrasta con las imágenes, los procesos reales y la suite; Windows LocalDB, Windows+Docker infra y Windows+Docker integral son **tres certificaciones diferenciadas**. El flujo Docker integral solo se marcará PASS tras ejecutar realmente `EJECUTAR_DOCKER_WINDOWS.ps1 -Accion levantar` en un Windows con Docker Desktop/WSL2. Haber creado este manual o superado CI Linux/Windows sin Docker Desktop **no sustituye** esa prueba física.